MedCore HMS

Multi-tenant, cloud-native Hospital Management Platform — Full-Stack Internship Project

Live application	https://medcore-hms-web.vercel.app
API documentation	https://medcore-api-production-b18a.up.railway.app/api/docs
Source code	https://github.com/LuckyLuck786/hospital_management_system
Author	Shaik Luqman

9 role-specific workspaces	**98** documented REST endpoints	**34** normalised data models	**90** automated tests (API, UI, browser)

1. Overview

MedCore HMS digitises the complete hospital workflow — registration, appointment scheduling, the doctor's encounter (vitals, diagnosis, e-prescription, lab orders), laboratory processing, pharmacy dispensing, ward and bed management, billing with online payments and insurance/TPA claims, and real-time notifications — in one secure, role-aware platform. Each hospital is an isolated tenant of a single SaaS deployment, so it serves anything from a 30-bed clinic to an 800-bed multi-speciality chain. All nine roles from the brief (Super Admin, Hospital Admin, Doctor, Nurse, Receptionist, Lab Technician, Pharmacist, Accountant, Patient) have their own dashboard and enforced API permissions.

2. Technology stack

Layer	Technology
Frontend	Next.js 15 (App Router), React 19, TypeScript, Tailwind CSS, TanStack Query, Zustand, React Hook Form + Zod, Recharts, Socket.IO client
Backend	NestJS 10, TypeScript, Prisma ORM, Passport JWT, BullMQ workers, Socket.IO gateway, Puppeteer PDF rendering, Multer + Sharp uploads
Data	PostgreSQL 16 (row-level multi-tenancy, AES-256-GCM encrypted clinical columns), Redis 7 (sessions, cache, queues)
Integrations	Razorpay (UPI / cards / netbanking + signed webhooks), SMTP / Resend email, Twilio SMS, Google Cloud Storage, optional Sentry
DevOps	Docker + Docker Compose, Nginx reverse proxy, GitHub Actions CI, Vercel (frontend), Railway (API, PostgreSQL, Redis)
Testing	Jest + Supertest integration tests on a real PostgreSQL database; Vitest + Testing Library component tests; Playwright browser end-to-end tests; live smoke test

3. Features delivered

Module	What is implemented	Status
Authentication & RBAC	Register, login, JWT access (15 min) + rotating refresh tokens in Redis per device, logout, email OTP verification, SMS phone OTP, forgot / reset password by emailed link, bcrypt-12 hashing, guards for all 9 roles, hospital (tenant) isolation on every request.	**Complete**
Hospital onboarding	Super Admin creates and verifies hospitals; admins manage departments, doctors, staff and user accounts.	**Complete**
Appointments	Weekly doctor schedule editor with configurable slot length, live slot availability (60-second Redis cache), booking by patient or reception, emergency bookings, status lifecycle PENDING → CONFIRMED → IN_PROGRESS → COMPLETED / CANCELLED / NO_SHOW, database-enforced double-booking protection for doctors and patients, 24 h and 1 h reminders.	**Complete**
Electronic Medical Records	Append-only encounters with vitals and auto-BMI, chief complaint, ICD-10 diagnosis, treatment plan, allergies, vaccination history, family-history flags and secure file attachments (PDF/images, 20 MB, private cloud storage with expiring links). Diagnosis and notes encrypted at rest.	**Complete**
Nurse station	Triage queue, vital-sign recording with trend charts and abnormal-value highlighting; the doctor's consultation form pre-fills from today's triage.	**Complete**
Prescriptions	Medicine search from live pharmacy stock, dosage / frequency (OD, BD, TDS, QID, SOS) / duration per item, professional PDF with hospital letterhead and doctor signature.	**Complete**
Laboratory	Test catalogue with gender-specific reference ranges; order → sample collected → processing → results → approval workflow; automatic abnormal flags; PDF reports visible to doctor and patient after approval.	**Complete**
Pharmacy	Inventory with batches (batch no., mfg / expiry, cost, MRP), receive stock, FIFO dispensing by earliest expiry, expired-stock quarantine, low-stock and nightly 30-day expiry alerts.	**Complete**
Wards & beds	Rooms with individual beds per department; admit a patient to a specific bed, one-step transfer to another bed (e.g. ward → ICU), discharge to cleaning, mark ready, take out of service; live occupancy KPIs and department occupancy heat map.	**Complete**
Billing & payments	One invoice per visit aggregating consultation, lab and pharmacy charges; finalise, collect cash / card / UPI at the counter or pay online through Razorpay with server-side signature verification and webhooks; partial payments; receipts.	**Complete**
Insurance / TPA	Cashless pre-authorisation of estimated treatment cost before admission (approve / partial / reject with validity date); claims against unpaid invoices linked to the pre-authorisation, insurer review, settlement posted directly on the invoice; patients notified and able to track claims.	**Complete**

Notifications	Real-time in-app notifications over Socket.IO, email and SMS channels via BullMQ fan-out workers for confirmations, reminders, lab reports, prescriptions, invoices, payments, low stock and emergencies.	**Complete**
Analytics & search	Role-specific dashboards with Recharts (revenue trend, appointment volume, patient growth, department load, bed occupancy), finance view with CSV export, global search across patients, doctors, medicines and appointments.	**Complete**
Patient portal	Book appointments, view medical records, vitals trends, vaccinations, lab reports and prescriptions, download PDFs, pay bills online and follow insurance claims.	**Complete**
Audit & settings	Audit-log viewer of every create / update / delete (user, IP, time, redacted request); profile editing and phone verification for every user.	**Complete**
UI / UX	Professional landing page, split-screen sign-in with one-click demo accounts, responsive layout with mobile navigation, loading / empty / error states throughout.	**Complete**

4. Architecture & key design decisions

- **Row-level multi-tenancy:** every hospital-owned table carries `hospitalId`; a Tenant guard pins each request to the caller's hospital and services re-check ownership, so guessing another hospital's record ID returns 403.
- **Request pipeline:** Helmet → rate limiting → strict CORS → JWT guard → Roles guard → Tenant guard → validation → service → Prisma; responses use one envelope `{ success, data, meta }` / `{ success:false, error:{ code, message } }`.
- **Concurrency:** partial unique indexes guarantee that only one of two simultaneous bookings for a slot succeeds (409 for the other) — safe across multiple API instances without row locks.
- **Caching:** Redis stores refresh tokens per device (`rt:{userId}:{deviceId}`), doctor availability for 60 s with invalidation on every booking, reminder de-duplication and BullMQ queues.
- **Data model:** 34 Prisma models normalised to 3NF with 8 migrations — weekly schedule templates, prescription line items, structured lab results with reference ranges, medicine batches, individual beds, pre-authorisations, insurance claims, vital signs, vaccinations and an audit log. Soft deletes on patients, doctors and appointments.
- **Deployment:** Next.js frontend on Vercel rewrites `/api` to the NestJS API on Railway (same origin, no CORS for REST); WebSockets connect directly. The API container runs database migrations on start-up. Docker Compose + Nginx reproduce the stack locally.

5. Security

- bcrypt (cost 12) password hashing
- 256-bit JWT secrets; refresh-token rotation with replay rejection
- 100 req / 15 min on auth routes, 1000 req / 15 min on the API
- Helmet headers, strict CORS allow-list, HTTPS everywhere
- SameSite=Strict httpOnly refresh cookie; Bearer tokens for writes
- AES-256-GCM encryption of diagnosis, notes and treatment plans
- Audit log for every write with redacted request bodies
- Log sanitiser — passwords, tokens and OTPs never logged
- Upload allow-list, magic-byte checks, private storage, signed URLs
- Ownership checks on users, schedules, stock, records and claims

6. Testing & quality

Suite	Coverage
API integration (Jest + Supertest)	52 tests in 10 suites against a real PostgreSQL database: tenancy isolation, patient data isolation, simultaneous-booking race, refresh-token replay, expired-stock dispensing, invoice arithmetic, Razorpay webhook signatures, lab / prescription / admin / onboarding / auth workflows, vitals, bed admission and transfer, pre-authorisation, insurance claims, audit log and encryption at rest. Backend line coverage 77%.

UI components (Vitest + Testing Library)	25 tests: login validation and demo accounts, vitals form, pre-authorisation and claim forms, role-based navigation, family-history toggles and shared components.
Browser end-to-end (Playwright)	13 journeys on desktop and mobile: sign-in and validation, forgot password, admin KPIs and audit log, doctor chart, pharmacist inventory, patient records and bills, nurse vitals, admit → transfer → discharge, claim review and filing, mobile navigation.
Continuous integration	GitHub Actions runs typechecks, all three test suites (the browser suite against a freshly migrated and seeded stack) and a production build on every push.
Live smoke test	Checks the deployed chain end to end: landing page, API proxy, logins, dashboards, invoices, Razorpay order, forged-signature rejection, lab and prescription PDFs — 12 / 12 passing.

7. Documentation in the repository

- `README.md` — setup, demo credentials, feature list and deployment guide
- `docs/ARCHITECTURE.md` — component and request-flow diagrams, design decisions
- `docs/ER_DIAGRAM.md` — full entity-relationship diagram generated from the Prisma schema
- `docs/PERMISSIONS_MATRIX.md` — all 98 endpoints × 9 roles, generated from the code
- `docs/PROJECT_REPORT.md` — technical decisions, challenges and lessons learned
- `docs/coverage/` — HTML test-coverage report; Swagger UI live at `/api/docs`

8. Demo credentials

All data is fictional. The sign-in page also has one-click buttons for each role.

Role	Email	Password
Hospital Admin	admin@medcore.com	Admin@123
Doctor	dr.rajesh.sharma@medcore.com	Doctor@123
Nurse	nurse@medcore.com	Nurse@123
Receptionist	reception@medcore.com	Reception@123
Lab Technician	labtech@medcore.com	LabTech@123
Pharmacist	pharmacist@medcore.com	Pharma@123
Accountant	accountant@medcore.com	Accountant@123
Patient	rahul.verma@email.com	Patient@123
Super Admin	superadmin@medcore.com	SuperAdmin@123